NB: Heg har brukt CHATGPT for generering av .csv dataen som er ment for denne oppgaven. Alt som er skrevet er mitt arbeid og har valgt å lage ny data og bruke det vi har jobbet med i timene for å utfordre meg litt grann.

Arbeidskrav 1 – Innføring i dataanalyse

Navn: Mark Tzvetoslavov

Dato: 25.10.2026

1. Oppgaven

For denne spesifikk oppgaven er å dokumentere at oppgaven vi har jobbet er et vel fungerende SQLite database og at den klarer rett og slett ta imot data og kunne representere denne. Databasen inneholde to tabeller med flere datatyper. I min database inneholder hver tabell en primærnøkkel og en fremmednøkkelrelasjon mellom tabellene.

I oppgaven har jeg også lagt til to SQL-spørringer som rett og slett viser den indre og den ytre sammenkobling.

2. Database og installasjon

Løsningen min bruker SQLite Jeg har lagt en forklaring på hvordna man laster ned denne og jobber med denne på README.md filen. Databasefilen heter arbeidskrav1.db og inneholder tabellene kunder og bestillinger. Begge tabellene har data.

For å dokumentere SQLite på min egen maskin kjører jeg denne spørringen:

```
SELECT sqlite_version() AS sqlite_versjon;
```

3. Beskrivelse av dataene

Dataene beskriver kunder og bestillinger i en fitkiv nettbutikk. Det jeg har valgt å gjøre er å lage min egen data, i stedenfor å bruke dataen som vi har jobbet i timene. Jeg tenker selv om denne dataen er generert av CHATGPT, så da den meg muligheten til å tenke for meg selv og gjøre alt fra scratch.

- kunder.csv: **200 kunder**.
- bestillinger.csv: **1 000 bestillinger**.

Av de 200 kundene har 170 minst én bestilling, mens 30 kunder ikke har noen bestillinger. Dette gjør det mulig å vise forskjellen mellom indre og ytre sammenkobling. Filene er kommaseparerte og bruker UTF-8. Datoene er skrevet som YYYY-MM-DD, og beløpene har punktum som desimalskilletegn og to desimaler i CSV-filene. I denne oppgaven tolkes beløpene som norske kroner og ikke USD

Tabellen kunder

Hver rad i tabellen beskriver 1kunde.

Kolonne	Datatype	Beskrivelse
kunde_id	INTEGER	Unikt kundenummer. Primærnøkkel i tabellen.
navn	TEXT	Kundens navn.
by	TEXT	Byen kunden er registrert i.
registrert_dato	TEXT	Datoen kunden ble registrert, skrevet som YYYY-MM-DD.

Tabellen bestillinger

Hver rad i tabellen beskriver 1 bestilling.

Kolonne	Datatype	Beskrivelse
bestilling_id	INTEGER	Unikt bestillingsnummer. Primærnøkkel i tabellen.
kunde_id	INTEGER	Kunden som har lagt inn bestillingen. Fremmednøkkel til kunder.kunde_id.
dato	TEXT	Bestillingsdato, skrevet som YYYY-MM-DD.
belop	REAL	Bestillingens beløp, tolket som norske kroner i denne oppgaven.
status	TEXT	Bestillingens status: Levert, Sendt eller Behandles.

Databasen bruker dermed INTEGER, TEXT og REAL. SQLite har ikke en egen lagringsklasse for datoer, så datoene er lagret som tekst i et konsekvent format. Dette kan da sammenliknes med andre språk som SQL.

4. Opprettelse av tabellene og nøklene

Tabellene er definert med følgende SQL:

```
PRAGMA foreign_keys = ON;

CREATE TABLE kunder (
   kunde_id INTEGER PRIMARY KEY,
   navn TEXT NOT NULL,
   by TEXT NOT NULL,
   registrert_dato TEXT NOT NULL
);

CREATE TABLE bestillinger (
   bestilling_id INTEGER PRIMARY KEY,
   kunde_id INTEGER NOT NULL,
   dato TEXT NOT NULL,
   belop REAL NOT NULL CHECK (belop >= 0),
   status TEXT NOT NULL
      CHECK (status IN ('Levert', 'Sendt', 'Behandles')),
   FOREIGN KEY (kunde_id) REFERENCES kunder(kunde_id)
);
```

kunde_id er primærnøkkelen i kunder, og bestilling_id er primærnøkkelen i bestillinger. Disse identifiserer hver kunde og hver bestilling.kunde_id i bestillingstabellen er fremmednøkkelen som peker til kundetabellen. Når fremmednøkkelkontrollen er aktivert, kan en bestilling ikke knyttes til et kundenummer som mangler i kunder.

Antall rader kan kontrolleres med:

```
SELECT 'kunder' AS tabell, COUNT(*) AS antall
FROM kunder

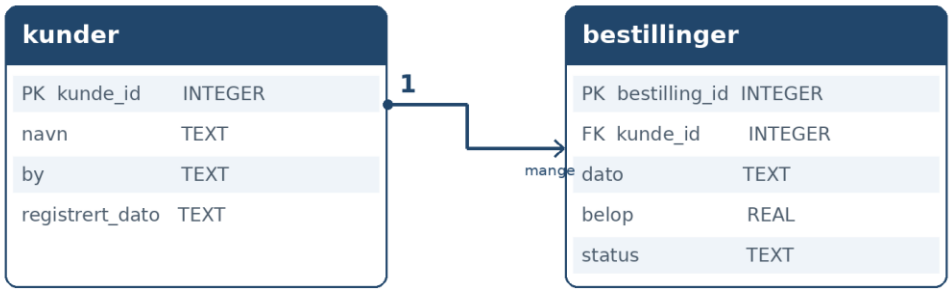
UNION ALL

SELECT 'bestillinger' AS tabell, COUNT(*) AS antall
FROM bestillinger;
```

Resultatet for den medfølgende databasen er:

Tabell	Antall rader
kunder	200
bestillinger	1 000

5. Tegning og forklaring av relasjonen



Koblingen går mellom kunder.kunde_id og bestillinger.kunde_id.

Relasjonen er **én-til-mange**:

- Én kunde kan ha ingen, én eller flere bestillinger.
- Hver bestilling tilhører nøyaktig én kunde.

I figuren betyr **PK** primærnøkkel og **FK** fremmednøkkel. 1 på kundesiden og 0..mange på bestillingssiden viser antallet rader som kan være knyttet til hverandre.

6. SQL spørringer

```sql
-- 1. INNER JOIN: Viser bare bestillinger som har en kunde tilknyttet dem. ---
SELECT
    k.kunde_id,
    k.navn,
    k.by,
    b.bestilling_id,
    b.dato,
    b.belop,
    b.status
FROM kunder AS k
LEFT JOIN bestillinger AS b
    ON k.kunde_id = b.kunde_id
ORDER BY k.kunde_id, b.bestilling_id;

-- 2. LEFT JOIN: Viser alle kunder i tillegg til kunder uten bestillinger. ---
SELECT
    k.kunde_id,
    k.navn,
    COUNT(b.bestilling_id) AS antall_bestillinger,
    COALESCE(ROUND(SUM(b.belop), 2), 0) AS totalbelop
FROM kunder AS k
LEFT JOIN bestillinger AS b
    ON k.kunde_id = b.kunde_id
GROUP BY k.kunde_id, k.navn
ORDER BY k.kunde_id;
```